AIR-GAPPED

MULTI-MODAL AGENTIC AI COMPLETE BUILD PLAN

10 Tasks · 50 Steps · CPU-First · Docker / Linux · Streamlit UI

Hardware: 40 GB RAM · CPU Only · Windows Host → Docker Linux Containers

Stack: Ollama · LangGraph · Qdrant · Postgres · Redis · FastAPI · Streamlit

PROJECT FOLDER STRUCTURE

Full reference — every file has a home before you write it

```
aiagent/
├── docker/
│   ├── Dockerfile.api
│   ├── Dockerfile.streamlit
│   └── docker-compose.yml
├── services/
│   ├── llm/
│   │   ├── router.py
│   │   ├── ollama_client.py
│   │   └── model_config.yaml
│   ├── rag/
│   │   ├── embedder.py
│   │   ├── retriever.py
│   │   ├── reranker.py
│   │   └── chunker.py
│   ├── vision/
│   │   ├── ocr.py
│   │   ├── image_qa.py
│   │   └── chart_analyzer.py
│   ├── video/
│   │   ├── frame_extractor.py
│   │   ├── video_summarizer.py
│   │   └── scene_detector.py
│   ├── audio/
│   │   ├── stt.py
│   │   ├── tts.py
│   │   └── command_parser.py
│   ├── memory/
│   │   ├── short_term.py
│   │   ├── long_term.py
│   │   └── context_manager.py
│   └── agents/
│       ├── orchestrator.py
│       ├── rag_agent.py
│       ├── vision_agent.py
│       ├── video_agent.py
│       ├── audio_agent.py
│       ├── memory_agent.py
│       └── analyst_agent.py
├── api/
│   ├── main.py
│   ├── routers/
│   │   ├── chat.py
│   │   ├── upload.py
│   │   └── agent.py
│   └── middleware/
│       ├── auth.py
│       └── logging.py
└── streamlit_app/
```

```
■   ■■■ app.py
■   ■■■ pages/
■   ■   ■■■ 01_Chat.py
■   ■   ■■■ 02_Documents.py
■   ■   ■■■ 03_Vision.py
■   ■   ■■■ 04_Video.py
■   ■   ■■■ 05_Voice.py
■   ■   ■■■ 06_Monitoring.py
■   ■■■ components/
■       ■■■ sidebar.py
■       ■■■ chat_ui.py
■       ■■■ file_uploader.py
■■■ db/
■   ■■■ qdrant_init.py
■   ■■■ postgres_init.sql
■   ■■■ redis_config.py
■■■ monitoring/
■   ■■■ prometheus.yml
■   ■■■ grafana/
■   ■   ■■■ dashboards/
■   ■■■ loki/
■       ■■■ config.yaml
■■■ eval/
■   ■■■ hallucination.py
■   ■■■ retrieval_quality.py
■   ■■■ agent_accuracy.py
■■■ config/
■   ■■■ settings.py
■   ■■■ logging.yaml
■■■ tests/
■   ■■■ test_rag.py
■   ■■■ test_vision.py
■   ■■■ test_agents.py
■■■ scripts/
■   ■■■ pull_models.sh
■   ■■■ init_db.sh
■■■ .env.example
■■■ README.md
```

All code lives under aiagent/. Docker context is the project root. Each service is independently importable. Streamlit pages map 1-to-1 with tasks.

TASK OVERVIEW & DEPENDENCY MAP

TASK	TITLE	DEPENDS ON	TAGS
T01	Docker Infrastructure & Project Skeleton	ALL tasks depend on this	Infrastructure, Docker
T02	LLM Layer — Model Router & Ollama Client	Task 1 (Docker)	LLM, Routing
T03	Vector DB & Agentic RAG Pipeline	Task 1 (Qdrant container)	RAG, Qdrant
T04	Vision AI — OCR, Image QA & Chart Analysis	Task 2 (LLaVA via router)	Vision, OCR
T05	Document Intelligence — PDF, DOCX, CSV, Excel	Task 3 (RAG ingest)	Documents, PDF
T06	Multi-Agent System with LangGraph Orchestration	Tasks 2-5 (all services)	Agents, LangGraph
T07	Memory System — Redis Short-Term + Postgres Long-Term	Task 1 (containers)	Memory, Redis
T08	Video AI — Frame Extraction, Summarization & Search	Task 4 (Vision for frames)	Video, CV
T09	Voice AI — STT, TTS & Voice Command Agent	Task 2 (LLM for command understanding)	Voice, STT
T10	Monitoring, Evaluation & LLMOps Dashboard	ALL tasks feed metrics into this layer	Monitoring, Observability

DETAILED TASK BREAKDOWN

Each task = 5 steps · Each step = files + instructions

TASK 01

Docker Infrastructure & Project Skeleton

GOAL

Set up the complete Docker environment, folder structure, base configs, and verify all services start correctly on CPU/Windows host running Linux containers.

TAGS

[Infrastructure] [Docker] [Foundation]

COMPATIBLE WITH

ALL tasks depend on this

STEP 1 · Install Docker Desktop & configure Linux mode

On Windows: install Docker Desktop, enable WSL2 backend, set RAM limit to 32 GB in Docker Desktop > Resources.
Verify: docker --version && docker compose version
Enable BuildKit: set DOCKER_BUILDKIT=1 in Windows environment variables.

STEP 2 · Create project skeleton directories

FILES CREATED

```
+ aiagent/ (full tree)
```

Run the mkdir commands below inside WSL2 terminal to create every folder shown in the folder tree.
mkdir -p aiagent/{docker,services/{llm,rag,vision,video,audio,memory,agents},api/{routers,middleware},streamlit_app/{pages,components},db,monitoring/{grafana/dashboards,loki},eval,config,tests,scripts}
This creates the full tree in one command. Verify with: find aiagent -type d | sort

STEP 3 · Write docker-compose.yml with all services

FILES CREATED

```
+ docker/docker-compose.yml
```

Define services: ollama, qdrant, postgres (with pgvector), redis, fastapi, streamlit, prometheus, grafana, loki.
Use CPU-friendly resource limits (mem_limit per service). Ollama gets 16 GB, FastAPI 4 GB, Streamlit 2 GB.
All services on internal bridge network aiagent_net. Only Streamlit port 8501 exposed to host.

STEP 4 · Write Dockerfiles for API and Streamlit

FILES CREATED

```
+ docker/Dockerfile.api
+ docker/Dockerfile.streamlit
```

Dockerfile.api: FROM python:3.11-slim, install FastAPI + langchain + qdrant-client + psycopg2 + redis.
Dockerfile.streamlit: FROM python:3.11-slim, install streamlit + plotly + pandas + requests.
Both images use multi-stage builds to keep size small. Copy only needed files.

STEP 5 · Boot stack & validate connectivity

FILES CREATED

```
+ config/settings.py
+ .env.example
+ scripts/pull_models.sh
```

docker compose up -d then check: docker compose ps (all services healthy).
Test Ollama: curl http://localhost:11434/api/tags
Test Qdrant: curl http://localhost:6333/healthz | Test Redis: docker exec -it redis redis-cli ping
Write config/settings.py with all service URLs, model names, and env-var overrides.

TASK 02

LLM Layer — Model Router & Ollama Client

GOAL

Build the intelligent model router that selects the correct local model (fast / reasoning / vision / summarization) for each task, with CPU-optimised GGUF quantization via Ollama.

TAGS

[LLM] [Routing] [Ollama] [CPU]

COMPATIBLE WITH

Task 1 (Docker) + Task 6 (Agents use router)

STEP 1 · Pull CPU-friendly GGUF models via Ollama

Run scripts/pull_models.sh which executes: ollama pull phi3:mini-q4, ollama pull mistral:7b-q4, ollama pull llava:7b-q4, ollama pull deepseek-r1:7b-q4.
Each model is ~4 GB at Q4 quantization. Total ~16 GB. Models stored in Ollama volume (persists across restarts).
Verify: ollama list should show all four models.

FILES CREATED

```
+ scripts/pull_models.sh
```

STEP 2 · Define model config YAML

Create model_config.yaml mapping task types to model names and parameters.
Keys: fast (phi3:mini), reasoning (deepseek-r1:7b), vision (llava:7b), summarization (mistral:7b), audio (faster-whisper).
Include per-model params: temperature, max_tokens, context_length, timeout_seconds.

FILES CREATED

```
+ services/llm/model_config.yaml
```

STEP 3 · Build Ollama async client

Write ollama_client.py using httpx.AsyncClient to call Ollama REST API (/api/generate, /api/chat).
Support streaming responses with async generator. Handle connection errors with retry (3 attempts, exponential backoff).
Add token-usage logging: log prompt_tokens, completion_tokens to Prometheus counter.

FILES CREATED

```
+ services/llm/ollama_client.py
```

STEP 4 · Implement model router logic

router.py: classify incoming task using keyword rules + optional small classifier call.
Task types: CHAT, REASON, VISION, SUMMARIZE, CODE, AUDIO. Each maps to a model from config.
Router returns (model_name, model_params) tuple. Falls back to 'fast' model on unknown type.

FILES CREATED

```
+ services/llm/router.py
```

STEP 5 · Expose router via FastAPI endpoint & Streamlit test page

Add POST /api/v1/llm/generate to api/routers/chat.py. Accepts {prompt, task_type, stream:bool}.
In streamlit_app/pages/01_Chat.py: text input + task-type dropdown + streaming response display.
Test all four models respond correctly. Measure latency; document expected tokens/sec on CPU.

FILES CREATED

```
+ api/routers/chat.py
+ streamlit_app/pages/01_Chat.py
```

TASK 03

Vector DB & Agentic RAG Pipeline

GOAL

Build the full retrieval pipeline: chunking, embedding, Qdrant storage, hybrid BM25+semantic search, reranking, and multi-hop retrieval for complex queries.

TAGS

[RAG] [Qdrant] [Embeddings] [Retrieval]

COMPATIBLE WITH

Task 1 (Qdrant container) + Task 2 (LLM for answer gen) + Task 5 (Docs)

STEP 1 · Initialize Qdrant collections

Write db/qdrant_init.py: create collections text_chunks (384-dim), image_embeddings (512-dim), video_frames (512-dim), memory_store (384-dim).
Use cosine distance. Enable payload indexing on fields: doc_id, source, timestamp, user_id.
Run script on startup via docker compose entrypoint. Idempotent (skip if collection exists).

FILES CREATED

```
+ db/qdrant_init.py
```

STEP 2 · Build chunker and embedder

chunker.py: recursive character splitter, chunk_size=512, overlap=64. Returns list of {text, metadata} dicts.
embedder.py: load sentence-transformers/all-MiniLM-L6-v2 locally (CPU-friendly, 90 MB). Batch embed with batch_size=32.
Both modules are synchronous (CPU); wrapped in asyncio.run_in_executor for async compatibility.

FILES CREATED

```
+ services/rag/chunker.py
+ services/rag/embedder.py
```

STEP 3 · Implement hybrid retriever (BM25 + semantic)

retriever.py: semantic search via Qdrant client, BM25 via rank_bm25 library against in-memory index.
Combine scores: final_score = 0.6 * semantic + 0.4 * bm25. Return top-K (default 5) chunks.
Support metadata filters: doc_id, date_range, user_id passed as Qdrant filter object.

FILES CREATED

```
+ services/rag/retriever.py
```

STEP 4 · Add reranker and multi-hop retrieval

reranker.py: use cross-encoder/ms-marco-MiniLM-L-2-v2 (lightweight, CPU). Rerank top-20 → return top-5.
Multi-hop: if query needs >1 entity, retriever runs iterative retrieval chain (max 3 hops), feeding prior results as context.
Example: 'employee attendance last quarter' → hop1: employee records, hop2: attendance logs, hop3: date filter.

FILES CREATED

```
+ services/rag/reranker.py
```

STEP 5 · RAG API endpoints & Documents page in Streamlit

Add POST /api/v1/rag/query and POST /api/v1/rag/ingest to api/routers endpoints.
streamlit_app/pages/02_Documents.py: file upload (PDF/DOCX/TXT/CSV), ingest button, RAG query box, retrieved chunks viewer with scores.
Show retrieval latency and chunk provenance (filename, page, score) in a table below each answer.

FILES CREATED

```
+ api/routers/upload.py
+ streamlit_app/pages/02_Documents.py
```

TASK 04

Vision AI — OCR, Image QA & Chart Analysis

GOAL

Build the vision pipeline: OCR for scanned docs/forms, image-based QA using LLaVA, screenshot debugging, and chart/diagram analysis, all running CPU-only.

TAGS

[Vision] [OCR] [LLaVA] [Multi-Modal]

COMPATIBLE WITH

Task 2 (LLaVA via router) + Task 3 (image embeddings in Qdrant) + Task 6 (Vision Agent)

STEP 1 · Set up OCR stack (Tesseract + PaddleOCR)

FILES CREATED

In Dockerfile.api add: apt-get install -y tesseract-ocr tesseract-ocr-eng tesseract-ocr-ara && pip install pytesseract paddleocr.
ocr.py: auto-selects Tesseract for simple scans, PaddleOCR for complex layouts/forms/invoices.
Pre-process images: grayscale → denoise → threshold before OCR. Returns structured text + bounding boxes.

```
+ services/vision/ocr.py
+ docker/Dockerfile.api
```

STEP 2 · Build image QA module with LLaVA

FILES CREATED

image_qa.py: encode image to base64, send to Ollama /api/generate with llava:7b model and user question.
Support multi-image input (up to 4 images per prompt). Stream response back.
Add image embedding: use CLIP (openai/clip-vit-base-patch32 via transformers) → store vector in Qdrant image_embeddings collection.

```
+ services/vision/image_qa.py
```

STEP 3 · Chart and diagram analyzer

FILES CREATED

chart_analyzer.py: detect chart type (bar/line/pie/table) using image classifier, then pass to LLaVA with structured prompt.
Extract numeric data from charts using pytesseract on axis labels. Return JSON with {chart_type, data_points, summary}.
For screenshots: detect UI elements, error dialogs, log text using OCR + LLaVA description.

```
+ services/vision/chart_analyzer.py
```

STEP 4 · Vision API endpoints

FILES CREATED

Add POST /api/v1/vision/ocr, /api/v1/vision/qa, /api/v1/vision/analyze to api/routers.
Accept multipart/form-data with image file + optional question text. Return JSON with text, answer, or chart data.
Add image semantic search: POST /api/v1/vision/search?q=text returns similar images from Qdrant.

```
+ api/routers/upload.py (extend)
```

STEP 5 · Vision page in Streamlit

FILES CREATED

streamlit_app/pages/03_Vision.py: tabs for OCR, Image QA, Chart Analysis, Screenshot Debug.
OCR tab: upload image → show extracted text with bounding box overlay.
Image QA: upload + question → streaming answer.
Chart tab: upload chart → display detected type, extracted data as st.dataframe, LLaVA summary.

```
+ streamlit_app/pages/03_Vision.py
```

TASK 05

Document Intelligence — PDF, DOCX, CSV, Excel

GOAL

Build the document ingestion pipeline supporting PDF (native + scanned), DOCX, TXT, CSV, and Excel. Integrate with OCR for scanned docs and feed into RAG.

TAGS

[Documents] [PDF] [OCR] [Ingestion]

COMPATIBLE WITH

Task 3 (RAG ingest) + Task 4 (OCR for scanned) + Task 6 (RAG Agent)

STEP 1 · PDF ingestion (native text + scanned)

FILES CREATED

+ services/rag/chunker.py (extend for PDF)

Use pdfplumber for native-text PDFs (extracts text + tables). Detect if page is scanned (text < 50 chars) → fall back to OCR pipeline from Task 4.
Extract tables as pandas DataFrames → serialize to markdown for RAG chunking.
Preserve metadata: filename, page_number, section_title (from headings) in chunk payload.

STEP 2 · DOCX, TXT, CSV, Excel ingestion

FILES CREATED

+ services/rag/chunker.py (extend)

DOCX: use python-docx to extract paragraphs + tables. Preserve heading hierarchy as metadata.
CSV/Excel: use pandas. Each row becomes a chunk. Column headers embedded in each chunk for context.
TXT: simple line-based chunker. All formats funnel into unified chunk format: {text, source, page, type}.

STEP 3 · Ingest pipeline orchestration

FILES CREATED

+ services/rag/embedder.py (extend)
+ db/postgres_init.sql

Write ingestion service: detect file type → route to correct loader → chunk → embed → upsert to Qdrant.
Async pipeline: file upload → background task (FastAPI BackgroundTasks) → progress events via SSE.
Deduplication: hash file content (SHA256), skip re-ingestion if hash already in postgres ingestion_log table.

STEP 4 · PostgreSQL metadata store

FILES CREATED

+ db/postgres_init.sql
+ db/postgres_client.py

postgres_init.sql: tables for documents (id, filename, hash, type, ingested_at, user_id), chunks (id, doc_id, page, text_preview), ingestion_log.
Enable pgvector extension for future long-term memory queries.
Write db/postgres_client.py with async connection pool (asyncpg). Used by ingest and memory services.

STEP 5 · Documents management UI in Streamlit

FILES CREATED

+ streamlit_app/pages/02_Documents.py (extend)

streamlit_app/pages/02_Documents.py (extend): document library table (filename, type, chunks, ingested_at), delete button, re-ingest button.
Upload widget: drag-and-drop multi-file, progress bar via polling /api/v1/ingest/status/{job_id}.
Filter by doc type, date. Click document → preview first 500 chars + chunk count.

TASK 06

Multi-Agent System with LangGraph Orchestration

GOAL

Build the orchestrator and all specialist agents (RAG, Vision, Video, Audio, Memory, Analyst) using LangGraph. Agents collaborate via message passing to complete complex multi-step tasks.

TAGS

[Agents] [LangGraph] [Orchestration] [Agentic AI]

COMPATIBLE WITH

Tasks 2-5 (all services) + Task 7 (Memory) + Task 8 (Video)

STEP 1 · Design LangGraph agent graph

Define StateGraph with nodes: orchestrator, rag_agent, vision_agent, video_agent, audio_agent, memory_agent, analyst_agent.
Shared state schema: {task_id, user_query, task_type, context, results, history, memory_key}.
Conditional edges from orchestrator → specialist agents based on task classification.

FILES CREATED

```
+ services/agents/orchestrator.py
```

STEP 2 · Implement specialist agents

Each agent is a LangGraph node (async function). Accepts state, calls its service, updates state.results.
rag_agent.py: calls retriever → reranker → LLM for answer generation.
vision_agent.py: calls OCR or image_qa based on subtask. analyst_agent.py: performs comparison, reasoning, report gen.

FILES CREATED

```
+ services/agents/rag_agent.py
+ services/agents/vision_agent.py
+ services/agents/analyst_agent.py
```

STEP 3 · Orchestrator routing logic

Orchestrator classifies query → builds execution plan (list of agents to invoke in order).
Supports parallel agent execution (LangGraph parallel branches) for independent subtasks.
After all agents complete, Analyst Agent synthesizes results into final response.

FILES CREATED

```
+ services/agents/orchestrator.py (complete)
```

STEP 4 · Agent API endpoint & streaming

POST /api/v1/agent/run: accepts {query, files[], context}. Returns task_id immediately.
GET /api/v1/agent/stream/{task_id}: SSE stream of agent events (agent_start, tool_call, agent_done, final_answer).
Log agent execution steps to postgres for audit trail.

FILES CREATED

```
+ api/routers/agent.py
```

STEP 5 · Agent execution viewer in Streamlit Chat page

streamlit_app/pages/01_Chat.py (extend): show agent trace panel (expandable). Each agent step shown as a card.
Display: which agent ran, what tool it called, latency, confidence score.
Final answer rendered with source citations (chunk filename + page). Thumbs up/down feedback stored to postgres.

FILES CREATED

```
+ streamlit_app/pages/01_Chat.py (extend)
+ streamlit_app/components/chat_ui.py
```

TASK 07

Memory System — Redis Short-Term + Postgres Long-Term

GOAL

Implement a two-tier memory system: Redis for session/short-term memory (conversation history, active context) and Postgres+pgvector for long-term semantic memory retrieval.

TAGS

[Memory] [Redis] [Postgres] [Context]

COMPATIBLE WITH

Task 1 (containers) + Task 6 (Memory Agent) + Task 3 (RAG context)

STEP 1 · Redis short-term memory

FILES CREATED

+ services/memory/short_term.py
+ db/redis_config.py

short_term.py: store conversation turns as Redis list with key session:{user_id}:{session_id}. TTL=2 hours.
Methods: add_turn(role, content), get_history(last_n=10), clear_session().
Serialize each turn as JSON: {role, content, timestamp, agent_used, tokens_used}.

STEP 2 · Postgres long-term memory

FILES CREATED

+ services/memory/long_term.py

long_term.py: store important facts/summaries with pgvector embeddings for semantic retrieval.
Table memory_store: (id, user_id, content, embedding vector(384), importance_score, created_at).
Methods: store_memory(content, user_id), retrieve_similar(query_embedding, top_k=5).

STEP 3 · Context manager & memory agent

FILES CREATED

+ services/memory/context_manager.py
+ services/agents/memory_agent.py

context_manager.py: builds prompt context by merging short-term history + long-term retrieved memories.
memory_agent.py: after each conversation turn, decides what to store in long-term (importance classifier).
Importance score: 0-1 float from small LLM call. Memories with score > 0.7 are persisted.

STEP 4 · User context & RBAC scaffolding

FILES CREATED

+ api/middleware/auth.py
+ db/postgres_init.sql (extend)

Add user_id to all memory keys and DB rows. Implement simple JWT auth middleware in api/middleware/auth.py.
RBAC: roles table in postgres. Permissions: can_upload, can_query, can_admin. Check permission on each API route.
Audit log table: every API call logged with user_id, endpoint, timestamp, response_code.

STEP 5 · Memory inspection UI in Streamlit

FILES CREATED

+ streamlit_app/components/sidebar.py

Add memory panel to Chat page sidebar: shows last 10 conversation turns from Redis.
Long-term memory viewer: search memories by semantic query, show importance scores, allow manual deletion.
Session management: new session button (clears Redis), view session history from postgres.

TASK 08

Video AI — Frame Extraction, Summarization & Search

GOAL

Build the full video processing pipeline: extract frames at scene boundaries, embed frames for semantic search, generate video summaries and event detection reports via the Video Agent.

TAGS

[Video] [CV] [Frame Analysis] [Scene Detection]

COMPATIBLE WITH

Task 4 (Vision for frames) + Task 3 (frame embeddings in Qdrant) + Task 6 (Video Agent)

STEP 1 · Frame extractor with scene detection

FILES CREATED

frame_extractor.py: use OpenCV (cv2) to extract frames. Use PySceneDetect for scene boundary detection.
Extract 1 key frame per scene + 1 frame/30s fallback. Save frames as JPEG with timestamp metadata.
Output: list of {frame_path, timestamp_sec, scene_id} dicts. Max 200 frames per video to stay CPU-friendly.

```
+ services/video/frame_extractor.py
```

STEP 2 · Frame embedding and Qdrant indexing

FILES CREATED

Use CLIP to embed each frame (same model as Task 4). Batch size 8 on CPU (~2s/batch).
Upsert to Qdrant video_frames collection with payload: {video_id, timestamp_sec, scene_id, frame_path}.
Also generate text description of each frame via LLaVA → embed text → store in text_chunks collection.

```
+ services/video/frame_extractor.py (extend)
```

STEP 3 · Video summarizer and event detector

FILES CREATED

video_summarizer.py: collect all frame descriptions → chunk into groups of 10 → summarize each group via Mistral → combine summaries.
scene_detector.py: classify each scene (person, machine, vehicle, text-on-screen, anomaly) using LLaVA zero-shot.
Output: structured report {video_id, duration, scenes[], events[], summary_text}.

```
+ services/video/video_summarizer.py
+ services/video/scene_detector.py
```

STEP 4 · Video Agent and API

FILES CREATED

video_agent.py: LangGraph node that receives video task → calls frame_extractor → summarizer → scene_detector → formats report.
POST /api/v1/video/analyze: upload video file → background task → return job_id.
GET /api/v1/video/search: semantic search across all indexed frames. Returns frame images + timestamps.

```
+ services/agents/video_agent.py
+ api/routers/upload.py (extend)
```

STEP 5 · Video analysis page in Streamlit

FILES CREATED

streamlit_app/pages/04_Video.py: upload video widget (max 500 MB), progress bar, scene timeline.
Results: video summary text, scene cards with thumbnail + description, event detection table.
Semantic search: text query → returns matching frame thumbnails with timestamp links.

```
+ streamlit_app/pages/04_Video.py
```

TASK 09

Voice AI — STT, TTS & Voice Command Agent

GOAL

Build offline Speech-to-Text (Faster-Whisper), Text-to-Speech (Piper/Coqui), and a voice command parser that routes spoken commands to the appropriate agent.

TAGS

[Voice] [STT] [TTS] [Whisper] [Offline]

COMPATIBLE WITH

Task 2 (LLM for command understanding) + Task 6 (Audio Agent) + Task 1 (Docker)

STEP 1 · Faster-Whisper STT setup	FILES CREATED
stt.py: load faster-whisper model (base or small, ~150 MB) on CPU with int8 quantization. transcribe(audio_bytes) → returns {text, language, segments[], word_timestamps}. Support WAV, MP3, OGG input. Convert to 16kHz mono via ffmpeg before passing to Whisper.	`+ services/audio/stt.py`

STEP 2 · Offline TTS with Piper	FILES CREATED
tts.py: integrate Piper TTS (CPU-native, real-time on modern CPU). Download en_US-lessac-medium model (~60 MB). synthesize(text) → returns WAV bytes. Cache synthesized audio in Redis (TTL=1h) to avoid re-synthesis. Fallback: use pyttsx3 if Piper unavailable. TTS disabled in headless/server mode unless requested.	`+ services/audio/tts.py`

STEP 3 · Voice command parser	FILES CREATED
command_parser.py: takes transcript text → extracts intent + entities using Phi-3 mini with structured prompt. Command intents: ANALYZE_DOC, SEARCH, SUMMARIZE, COMPARE, OPEN_RECORD. Entities: document name, date, person. Returns {intent, entities, confidence, raw_text} → passed to orchestrator as structured task.	`+ services/audio/command_parser.py`

STEP 4 · Audio Agent and API	FILES CREATED
audio_agent.py: LangGraph node. Receives audio task → STT → command_parser → route to target agent → TTS response. POST /api/v1/audio/transcribe: upload audio → return transcript. POST /api/v1/audio/command: upload audio → full agent pipeline → return {transcript, intent, answer, audio_response}.	`+ services/agents/audio_agent.py` `+ api/routers/agent.py (extend)`

STEP 5 · Voice UI in Streamlit	FILES CREATED
streamlit_app/pages/05_Voice.py: browser microphone recorder (st.audio + JS bridge) or file upload. Show: transcript, detected intent + entities, agent response text, play TTS audio button. Voice command history: last 20 commands with intent + result in a collapsible table.	`+ streamlit_app/pages/05_Voice.py`

TASK 10

Monitoring, Evaluation & LLMOps Dashboard

GOAL

Deploy Prometheus + Grafana + Loki for full observability. Build eval suite for hallucination rate, retrieval quality, and agent routing accuracy. Expose LLMOps metrics in Streamlit.

TAGS

[Monitoring] [Observability] [Evaluation] [LLMOps]

COMPATIBLE WITH

ALL tasks feed metrics into this layer

STEP 1 · Prometheus metrics instrumentation

FILES CREATED

Add prometheus_client to FastAPI. Expose /metrics endpoint.
Define counters/histograms: llm_requests_total, llm_latency_seconds, retrieval_precision, agent_routing_errors, tokens_used_total.
monitoring/prometheus.yml: scrape configs for fastapi:8000/metrics, qdrant:6333/metrics, ollama:11434/metrics.

```
+ monitoring/prometheus.yml
+ api/main.py (extend)
```

STEP 2 · Grafana dashboards

FILES CREATED

Pre-build 3 dashboard JSON files: System Overview (CPU, RAM, latency), LLM Ops (token usage, model distribution, error rate), RAG Quality (retrieval precision, reranker scores).
Mount dashboards as volume in docker-compose so they load automatically on first boot.
Grafana provisioning: datasources.yaml (Prometheus + Loki URLs), dashboards.yaml (dashboard folder path).

```
+ monitoring/grafana/dashboards/*.json
```

STEP 3 · Loki log aggregation

FILES CREATED

loki/config.yaml: single-process mode, filesystem storage. Retention 7 days.
Add Loki logging handler to Python logging config. Structured JSON logs with fields: service, level, user_id, task_id, duration_ms.
Grafana Loki datasource: explore logs by service, filter by user_id or task_id.

```
+ monitoring/loki/config.yaml
+ config/logging.yaml
```

STEP 4 · Evaluation suite

FILES CREATED

eval/hallucination.py: sample 50 RAG answers, score faithfulness vs retrieved chunks using NLI model (cross-encoder/nli-MiniLM2-L6-H768).
eval/retrieval_quality.py: measure precision@5 and MRR on labeled QA pairs.
eval/agent_accuracy.py: test orchestrator routing on 30 labeled queries, report confusion matrix per agent type.

```
+ eval/hallucination.py
+ eval/retrieval_quality.py
+ eval/agent_accuracy.py
```

STEP 5 · LLMOps monitoring page in Streamlit

FILES CREATED

streamlit_app/pages/06_Monitoring.py: embed Grafana panels via iframe OR fetch Prometheus metrics directly via API and render with plotly.
Show: real-time token usage gauge, latency p50/p95 line chart, top agents used pie chart, retrieval precision trend.
Eval results table: run eval suite button → display hallucination rate, retrieval precision, routing accuracy with pass/fail badges.

```
+ streamlit_app/pages/06_Monitoring.py
```

CROSS-TASK COMPATIBILITY MATRIX

	T01	T02	T03	T04	T05	T06	T07	T08	T09	T10
T01	■	■	■	■	■	■	■	■	■	■
T02		■	→			■			■	■
T03		→	■	→	→	■	→	→		■
T04		■	→	■		■		■		■
T05			■	■	■	■				■
T06		■	■	■	■	■	■	■	■	■
T07			→			■	■			■
T08			■	■		■		■		■
T09		■				■			■	■
T10										■

■ = Direct dependency / produces output consumed by → = Indirect / data flow

RECOMMENDED IMPLEMENTATION ORDER

PHASE 1 — Foundation (Week 1-2)	Task 01: Docker + Skeleton · Task 02: LLM Router + Ollama
PHASE 2 — Core AI (Week 3-4)	Task 03: RAG Pipeline · Task 05: Document Ingestion
PHASE 3 — Multi-Modal (Week 5-6)	Task 04: Vision AI · Task 08: Video AI · Task 09: Voice AI
PHASE 4 — Agents + Memory (Week 7-8)	Task 06: Multi-Agent System · Task 07: Memory System
PHASE 5 — Observability (Week 9-10)	Task 10: Monitoring + Eval